

seafarer_debriefs_migration.sql - AI-Assisted Enhancement Report

File: 04-migration-scripts/crewing/seafarer_debriefs_migration.sql

Purpose: Migrate synergy_seafarer.public.appraisal_debrief rows into smac_crewing_migration.shore.seafarer_debriefs with seafarer/vessel FK resolution, audit metadata, and migration.table_mappings records (levels/members are separate scripts).

1. Original Code

Baseline: git commit aa44a845 (~495 lines, single-step debrief migration with broken joins)

```
-- =====
-- Migration Script: seafarer_debriefs (with levels and level_members)
-- Legacy: synergy_seafarer.public.appraisal_debrief -> New: smac_crewing_migration.shore.seafarer_debriefs, seafarer_debrief_levels,
seafarer_debrief_level_members
-- Note: One source record splits into:
--   1. One row in seafarer_debriefs (main record)
--   2. Multiple rows in seafarer_debrief_levels (one per feedback array element)
--   3. Multiple rows in seafarer_debrief_level_members (one per debriefer in each feedback element)
--
-- DBLINK CONNECTIONS:
--   This script uses the following dblink connections (defined in 07-orchestration/00_common/connection_setup.sql):
--   1. 'synergy_seafarer' - Uses SS_HOST, SS_PORT, SS_DB, SS_USER, SS_PASS variables
--   2. 'synergy_vessel' - Uses SV_HOST, SV_PORT, SV_DB, SV_USER, SV_PASS variables
--   3. 'smac_master_migration' - Uses SMM_HOST, SMM_PORT, SMM_DB, SMM_USER, SMM_PASS variables (conditional)
--
-- Connection strings are built in connection_setup.sql using format():
--   format('host=%s port=%s dbname=%s user=%s password=%s', : 'SS_HOST', : 'SS_PORT', ...)
--
-- Variables are set by the PowerShell orchestration script (migrate_smac_crewing.ps1) from
-- migration_config_smac_crewing.json configuration file.
--
-- NOTE: All dblink queries use single-quoted strings. Single quotes inside queries are escaped as ''.
-- =====

\set ON_ERROR_STOP on
\timing on

\echo '=====
Migration: seafarer_debriefs (with levels and level_members)
=====

BEGIN;

-- Pre-migration checks
\echo 'Pre-migration checks...'

-- Verify dblink connections are established
\echo 'Verifying dblink connections...'
DO $$
DECLARE
    v_connections TEXT[];
    v_synergy_seafarer_ok BOOLEAN := false;
    v_synergy_vessel_ok BOOLEAN := false;
    v_smac_master_ok BOOLEAN := false;
BEGIN
    -- Get all active dblink connections
    SELECT dblink_get_connections() INTO v_connections;

    -- Check each required connection
    v_synergy_seafarer_ok := 'synergy_seafarer' = ANY(v_connections);
    v_synergy_vessel_ok := 'synergy_vessel' = ANY(v_connections);
    v_smac_master_ok := 'smac_master_migration' = ANY(v_connections);

    RAISE NOTICE '=====';
    RAISE NOTICE 'Dblink Connection Status: ';
    RAISE NOTICE '=====';
    RAISE NOTICE 'synergy_seafarer: %', CASE WHEN v_synergy_seafarer_ok THEN 'CONNECTED' ELSE 'NOT CONNECTED' END;
    RAISE NOTICE 'synergy_vessel: %', CASE WHEN v_synergy_vessel_ok THEN 'CONNECTED' ELSE 'NOT CONNECTED' END;
    RAISE NOTICE 'smac_master_migration: %', CASE WHEN v_smac_master_ok THEN 'CONNECTED' ELSE 'NOT CONNECTED' END;
    RAISE NOTICE '=====';

    -- Test connections with simple queries
    IF v_synergy_seafarer_ok THEN
        BEGIN
            PERFORM dblink('synergy_seafarer', 'SELECT 1');
            RAISE NOTICE 'synergy_seafarer connection test: OK';
        EXCEPTION WHEN OTHERS THEN
            RAISE WARNING 'synergy_seafarer connection test FAILED: %', SQLERRM;
        END;
    END;
```

```

END IF;

IF v_synergy_vessel_ok THEN
BEGIN
    PERFORM dblink('synergy_vessel', 'SELECT 1');
    RAISE NOTICE 'synergy_vessel connection test: OK';
EXCEPTION WHEN OTHERS THEN
    RAISE WARNING 'synergy_vessel connection test FAILED: %', SQLERRM;
END;
END IF;

IF v_smac_master_ok THEN
BEGIN
    PERFORM dblink('smac_master_migration', 'SELECT 1');
    RAISE NOTICE 'smac_master_migration connection test: OK';
EXCEPTION WHEN OTHERS THEN
    RAISE WARNING 'smac_master_migration connection test FAILED: %', SQLERRM;
END;
END IF;

-- Warn if critical connections are missing
IF NOT v_synergy_seafarer_ok THEN
    RAISE WARNING 'CRITICAL: synergy_seafarer connection not established. Check connection_setup.sql and SS_* variables.';
END IF;

IF NOT v_synergy_vessel_ok THEN
    RAISE WARNING 'CRITICAL: synergy_vessel connection not established. Check connection_setup.sql and SV_* variables.';
END IF;

IF NOT v_smac_master_ok THEN
    RAISE WARNING 'WARNING: smac_master_migration connection not established. Check connection_setup.sql and SMM_* variables.';
END IF;
END $$;

DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM information_schema.tables WHERE table_schema='shore' AND table_name='seafarer_debriefs') THEN
        RAISE EXCEPTION 'Target table shore.seafarer_debriefs does not exist. Please create the table schema first.';
    END IF;
    IF NOT EXISTS (SELECT 1 FROM information_schema.tables WHERE table_schema='shore' AND table_name='seafarer_debrief_levels') THEN
        RAISE EXCEPTION 'Target table shore.seafarer_debrief_levels does not exist. Please create the table schema first.';
    END IF;
    IF NOT EXISTS (SELECT 1 FROM information_schema.tables WHERE table_schema='shore' AND table_name='seafarer_debrief_level_members') THEN
        RAISE EXCEPTION 'Target table shore.seafarer_debrief_level_members does not exist. Please create the table schema first.';
    END IF;

    -- Verify prerequisite tables are migrated
    IF NOT EXISTS (SELECT 1 FROM migration.table_mappings WHERE target_table = 'seafarers' AND target_db = current_database()) THEN
        RAISE EXCEPTION 'seafarers table must be migrated first';
    END IF;
END $$;

\echo 'Legacy row count:'
DO $$
DECLARE
    v_legacy_count BIGINT;
BEGIN
    SELECT cnt INTO v_legacy_count FROM dblink('synergy_seafarer',
        'SELECT COUNT(*) FROM public.appraisal_debrief'
    ) AS t(cnt bigint);
    RAISE NOTICE 'Legacy appraisal_debrief table row count: %', v_legacy_count;

    IF v_legacy_count = 0 THEN
        RAISE WARNING 'WARNING: No rows found in legacy appraisal_debrief table!';
    END IF;
END $$;

\echo 'Sample legacy data (first 5 rows):'
DO $$
DECLARE
    r RECORD;
    v_row_count INT := 0;
BEGIN
    FOR r IN
        SELECT id, seafarer_uuid, vessel_uuid, vessel_category_id, debrief_status
        FROM dblink('synergy_seafarer',
            'SELECT id, seafarer_uuid, vessel_uuid, vessel_category_id, debrief_status FROM public.appraisal_debrief LIMIT 5'
        ) AS t(id uuid, seafarer_uuid uuid, vessel_uuid uuid, vessel_category_id bigint, debrief_status text)
    LOOP
        v_row_count := v_row_count + 1;
        RAISE NOTICE 'Row %: id=%, seafarer_uuid=%, vessel_uuid=%, vessel_category_id=%, debrief_status=%',
            v_row_count, r.id, r.seafarer_uuid, r.vessel_uuid, r.vessel_category_id, r.debrief_status;
    END LOOP;
END $$;

```

```

    IF v_row_count = 0 THEN
        RAISE WARNING 'WARNING: No sample rows returned from dblink query!';
    END IF;
END $$;

\echo 'Target table current counts:'
SELECT COUNT(*) AS debriefs_count FROM shore.seafarer_debriefs;
SELECT COUNT(*) AS levels_count FROM shore.seafarer_debrief_levels;
SELECT COUNT(*) AS members_count FROM shore.seafarer_debrief_level_members;

-- Check for existing data warning
DO $$
DECLARE
    v_existing_count BIGINT;
BEGIN
    SELECT COUNT(*) INTO v_existing_count FROM shore.seafarer_debriefs;
    IF v_existing_count > 0 THEN
        RAISE WARNING 'Target tables contain existing records. These will be deleted!';
    END IF;
END $$;

-- Clear existing data from target tables
TRUNCATE TABLE shore.seafarer_debrief_level_members;
TRUNCATE TABLE shore.seafarer_debrief_levels;
TRUNCATE TABLE shore.seafarer_debriefs;
-- Delete mappings from migration.table_mappings
DELETE FROM migration.table_mappings
WHERE target_table IN ('seafarer_debriefs', 'seafarer_debrief_levels', 'seafarer_debrief_level_members')
    AND target_db = current_database();

-- Create lookup tables for foreign key resolution
\echo 'Creating lookup tables for foreign key resolution...'

-- Create seafarer_id lookup mapping (source has seafarer_uuid as uuid, target needs uuid from mapping)
-- Note: seafarer_uuid is a UUID, so we should match directly with seafarers.id if UUIDs were preserved
-- Strategy: Direct UUID match (primary) - seafarer_uuid should match seafarers.id directly
-- Fallback: Use mapping table only for UUID source_ids (if seafarers preserved UUIDs in mapping table)
CREATE TEMP TABLE seafarer_uuid_mapping AS
SELECT
    source_id::uuid as legacy_uuid,
    target_id as new_id
FROM migration.table_mappings
WHERE target_table = 'seafarers'
    AND target_db = current_database()
    AND source_id ~ '^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$'; -- UUID pattern

-- Vessel: legacy vessel_id (bigint) -> smac vessel.vessels.id via smac_master migration.table_mappings;
-- debrief_vessel_lookup: appraisal_debrief.vessel_uuid = vessel_details.identifier -> legacy vessel_id;
-- vessel_revision_mapping: active revision per vessel (status=0, newest by created_at)

CREATE TEMP TABLE vessel_id_mapping AS
SELECT
    source_id::bigint AS legacy_id,
    target_id AS new_id
FROM dblink('smac_master_migration',
    'SELECT source_id, target_id FROM migration.table_mappings WHERE target_table = ''vessels'' AND source_id ~ ''^[0-9]+$''')
) AS t(source_id text, target_id uuid)
WHERE source_id IS NOT NULL AND target_id IS NOT NULL;

CREATE INDEX IF NOT EXISTS idx_vessel_id_mapping_legacy_id ON vessel_id_mapping(legacy_id);

-- Bridge vessel_uuid to synergy_vessel legacy vessel_id (bigint) for vessel_id_mapping
CREATE TEMP TABLE debrief_vessel_lookup AS
SELECT DISTINCT
    vd.identifier AS legacy_vessel_identifier,
    vd.vessel_id AS legacy_vessel_id
FROM dblink('synergy_vessel',
    'SELECT identifier, vessel_id FROM public.vessel_details WHERE identifier IS NOT NULL AND vessel_id IS NOT NULL')
) AS vd(identifier uuid, vessel_id bigint);

CREATE INDEX IF NOT EXISTS idx_debrief_vessel_lookup_identifier ON debrief_vessel_lookup(legacy_vessel_identifier);

\echo 'Creating vessel_revision_mapping for active revisions (seafarer_reliefs pattern)...'
CREATE TEMP TABLE vessel_revision_mapping AS
SELECT DISTINCT ON (vr.vessel_id)
    vr.vessel_id AS new_vessel_id,
    vr.id AS active_revision_id
FROM dblink('smac_master_migration',
    'SELECT id, vessel_id, status, created_at
    FROM vessel.vessel_revisions
    WHERE status = 0
    ORDER BY vessel_id, created_at DESC')

```

```

) AS vr(id uuid, vessel_id uuid, status integer, created_at timestamp)
INNER JOIN (
    SELECT new_id AS vessel_id FROM vessel_id_mapping
) AS all_vessels ON all_vessels.vessel_id = vr.vessel_id
ORDER BY vr.vessel_id, vr.created_at DESC;

\echo 'Vessel revision mapping row count:'
SELECT COUNT(*) AS vessel_revision_lookup_count FROM vessel_revision_mapping;

\echo 'Vessel mapping diagnostic (debrief_vessel_lookup + vessel_id_mapping + vessel_revision_mapping):'
DO $$
DECLARE
    v_lookup_count BIGINT;
    v_vid_count BIGINT;
    v_vrev_count BIGINT;
BEGIN
    SELECT COUNT(*) INTO v_lookup_count FROM debrief_vessel_lookup;
    SELECT COUNT(*) INTO v_vid_count FROM vessel_id_mapping;
    SELECT COUNT(*) INTO v_vrev_count FROM vessel_revision_mapping;
    RAISE NOTICE 'debrief_vessel_lookup rows: % (identifier to legacy vessel_id)', v_lookup_count;
    RAISE NOTICE 'vessel_id_mapping rows: %', v_vid_count;
    RAISE NOTICE 'vessel_revision_mapping rows: %', v_vrev_count;
END $$;

-- Create vessel_type_id lookup mapping (vessel_category_id bigint -> vessel_type_id uuid)
-- Strategy: Map vessel_category_id (bigint) from appraisal_debrief to categories.id (uuid) using migration.table_mappings
-- Note: categories table was migrated from vessel_categories, so source_id = vessel_categories.id (bigint), target_id = categories.id
(uuid)
CREATE TEMP TABLE vessel_type_id_mapping AS
SELECT
    source_id::bigint as legacy_category_id,
    target_id as new_type_id
FROM dblink('smac_master_migration',
    'SELECT source_id, target_id FROM migration.table_mappings WHERE target_table ILIKE ''categories'''
) AS t(source_id text, target_id uuid);

-- =====
-- STEP 1: Migrate seafarer_debriefs (main table)
-- =====
\echo 'Step 1: Migrating seafarer_debriefs data...'

-- Test dblink query first
\echo 'Testing dblink query...'
DO $$
DECLARE
    v_test_count BIGINT;
BEGIN
    SELECT COUNT(*) INTO v_test_count FROM dblink('synergy_seafarer',
        'SELECT COUNT(*) FROM public.appraisal_debrief'
    ) AS t(cnt bigint);
    RAISE NOTICE 'Dblink test query returned: % rows', v_test_count;

    IF v_test_count IS NULL THEN
        RAISE EXCEPTION 'ERROR: dblink query returned NULL. Check connection!';
    END IF;
END $$;

INSERT INTO shore.seafarer_debriefs (
    id,
    seafarer_id,
    seafarer_assignment_id,
    appraisal_id,
    reason_id,
    reason_text,
    vessel_id,
    vessel_type_id,
    sign_on_date,
    sign_off_date,
    appraisal_reports_available,
    attachments,
    current_stage,
    workflow_status,
    initiated_by,
    initiated_at,
    closed_by,
    closed_at,
    status,
    tenant_id,
    created_at,
    updated_at,
    archived_at,
    deleted_at,
    audit_info,

```

```

        vessel_revision_id
    )
SELECT
    legacy_data.id as id, -- Preserve legacy UUID id directly
    legacy_data.seafarer_uuid as seafarer_id, -- Map seafarer_uuid to seafarer_id with fallbacks
    NULL as seafarer_assignment_id, -- Not available in source
    NULL as appraisal_id, -- Not available in source
    legacy_data.debrief_reason_id as reason_id, -- Direct UUID mapping
    TRIM(legacy_data.other_debrief_reason) as reason_text, -- Direct copy
    vessel_map.new_id as vessel_id, -- legacy vessel_id (via debrief_vessel_lookup) -> smac vessels.id
    COALESCE(vessel_type_map.new_type_id, '00000000-0000-0000-0000-000000000000'::uuid) as vessel_type_id, -- Map vessel_category_id to
vessel_type_id via mapping, default to empty UUID if not found
    legacy_data.from_date::date as sign_on_date, -- Convert timestamp to date
    legacy_data.to_date::date as sign_off_date, -- Convert timestamp to date
    false as appraisal_reports_available, -- Default to false (NOT NULL)
    CASE
        WHEN legacy_data.attachments IS NULL OR array_length(legacy_data.attachments, 1) IS NULL THEN NULL
        ELSE to_jsonb(legacy_data.attachments)
    END as attachments, -- Convert text[] to jsonb
    -- current_stage: map SAC appraisal_debrief.debrief_status to SMAC stage values
    CASE LOWER(TRIM(COALESCE(legacy_data.debrief_status, '')))
        WHEN 'debriefing_initiated' THEN 'DebriefInitiated'
        WHEN 'ncs_review_1_initiated' THEN 'NCSReview1Initiated'
        WHEN 'ncs_review_1_recommended' THEN 'NCSReview1Recommended'
        WHEN 'ncs_review_2_recommended' THEN 'NCSReview2Recommended'
        WHEN 'debriefing_closed' THEN 'Closed'
        WHEN 'draft' THEN 'Draft'
        ELSE COALESCE(NULLIF(TRIM(legacy_data.debrief_status), ''), 'Draft')
    END as current_stage,
    -- Extract first feedback element's status for workflow_status
    CASE
        WHEN legacy_data.feedback IS NOT NULL
            AND jsonb_typeof(legacy_data.feedback) = 'array'
            AND jsonb_array_length(legacy_data.feedback) > 0
            AND legacy_data.feedback->0->'status' IS NOT NULL
        THEN legacy_data.feedback->0->'status'
        ELSE NULL
    END as workflow_status,
    NULL::uuid as initiated_by, -- User mapping not available, set to NULL
    COALESCE(legacy_data.initiated_date, legacy_data.created_at) as initiated_at, -- Prefer initiated_date, fallback to created_at
    NULL::uuid as closed_by, -- User mapping not available, set to NULL
    CASE
        WHEN LOWER(TRIM(COALESCE(legacy_data.debrief_status, ''))) IN ('debriefing_closed', 'closed', 'completed', 'finished') THEN
legacy_data.updated_at
        ELSE NULL
    END as closed_at, -- Set when debrief is closed (SAC debriefing_Closed or legacy closed states)
    'Active' as status, -- Always Active per migration rules (NOT NULL)
    'DEFAULT_TENANT_ID'::uuid as tenant_id, -- See constants.sql for DEFAULT_TENANT_ID
    COALESCE(legacy_data.created_at, NOW()) as created_at, -- Default to current timestamp
    COALESCE(legacy_data.updated_at, NOW()) as updated_at, -- Default to current timestamp
    NULL as archived_at, -- Not in source
    legacy_data.deleted_at as deleted_at, -- Direct copy
    jsonb_build_object(
        -- SMAC audit_info structure
        'created_by', CASE WHEN legacy_data.created_by_id IS NOT NULL AND legacy_data.created_by_id::text <> '' THEN
legacy_data.created_by_id::text ELSE NULL END,
        'deleted_by', CASE WHEN legacy_data.deleted_by IS NOT NULL AND legacy_data.deleted_by::text <> '' THEN legacy_data.deleted_by::text
ELSE NULL END,
        'updated_by', CASE WHEN legacy_data.updated_by_id IS NOT NULL AND legacy_data.updated_by_id::text <> '' THEN
legacy_data.updated_by_id::text ELSE NULL END,
        'archived_by', NULL,
        'submitted_by', NULL,
        'approved_at', NULL,
        'approved_by', NULL,
        'approval_notes', NULL,
        'rejected_by', NULL,
        'notes', NULL,
        -- Legacy migration metadata (preserved for reference)
        'legacy_id', legacy_data.id::text
    ) as audit_info,
    COALESCE(vrm.active_revision_id, '00000000-0000-0000-0000-000000000000'::uuid) as vessel_revision_id
FROM dblink('synergy_seafarer',
'SELECT
    id,
    seafarer_uuid,
    initiated_date,
    debrief_reason_id,
    other_debrief_reason,
    rank_id,
    vessel_category_id,
    vessel_uuid,
    attachments,
    debrief_status,

```

```

        from_date,
        to_date,
        mark_for_deactivation,
        is_manual,
        training_needs,
        feedback,
        deleted_at,
        deleted_by,
        created_by_id,
        created_by_name,
        created_at,
        updated_by_id,
        updated_by_name,
        updated_at
    FROM public.appraisal_debrief
    WHERE feedback IS NOT NULL
        AND jsonb_typeof(feedback) = 'array'
        AND jsonb_array_length(feedback) > 0'
) AS legacy_data(
    id uuid,
    seafarer_uuid uuid,
    initiated_date timestamp without time zone,
    debrief_reason_id uuid,
    other_debrief_reason text,
    rank_id bigint,
    vessel_category_id bigint,
    vessel_uuid uuid,
    attachments text[],
    debrief_status text,
    from_date timestamp without time zone,
    to_date timestamp without time zone,
    mark_for_deactivation boolean,
    is_manual boolean,
    training_needs jsonb,
    feedback jsonb,
    deleted_at timestamp without time zone,
    deleted_by text,
    created_by_id text,
    created_by_name varchar,
    created_at timestamp without time zone,
    updated_by_id text,
    updated_by_name varchar,
    updated_at timestamp without time zone
)
LEFT JOIN debrief_vessel_lookup v_bridge ON v_bridge.legacy_vessel_identifier = legacy_data.vessel_uuid
LEFT JOIN vessel_id_mapping vessel_map ON vessel_map.legacy_id = v_bridge.legacy_vessel_id
LEFT JOIN vessel_revision_mapping vrm ON vrm.new_vessel_id = vessel_map.new_id
LEFT JOIN vessel_type_id_mapping vessel_type_map ON vessel_type_map.legacy_category_id = legacy_data.vessel_category_id
WHERE vessel_map.new_id IS NOT NULL;
-- vessel_type_id via vessel_type_id mapping (can be default UUID when unmapped)

-- Check insert count immediately
\echo 'Checking insert count...'
DO $$
DECLARE
    v_inserted_count BIGINT;
BEGIN
    SELECT COUNT(*) INTO v_inserted_count FROM shore.seafarer_debriefs;
    RAISE NOTICE 'Rows inserted so far: %', v_inserted_count;

    IF v_inserted_count = 0 THEN
        RAISE WARNING 'WARNING: INSERT completed but no rows found in target table!';
        RAISE WARNING 'This could indicate:';
        RAISE WARNING ' 1. Source table is empty';
        RAISE WARNING ' 2. INSERT failed silently (check constraints/errors)';
        RAISE WARNING ' 3. Transaction rolled back';
    END IF;
END $$;

-- Post-migration validation
\echo 'Post-migration validation...'
DO $$
DECLARE
    v_inserted_count BIGINT;
    v_legacy_count BIGINT;
BEGIN
    SELECT COUNT(*) INTO v_inserted_count FROM shore.seafarer_debriefs;

    SELECT cnt INTO v_legacy_count FROM dblink('synergy_seafarer',
        'SELECT COUNT(*) FROM public.appraisal_debrief'
    ) AS t(cnt bigint);

    RAISE NOTICE '=====';

```

```

RAISE NOTICE 'Migration Summary: ';
RAISE NOTICE '=====';
RAISE NOTICE 'Legacy rows: %', v_legacy_count;
RAISE NOTICE 'Inserted rows: %', v_inserted_count;

IF v_inserted_count = 0 AND v_legacy_count > 0 THEN
    RAISE WARNING 'ERROR: No rows inserted but legacy table has % rows!', v_legacy_count;
ELSIF v_inserted_count != v_legacy_count THEN
    RAISE WARNING 'WARNING: Row count mismatch! Legacy: %, Inserted: %', v_legacy_count, v_inserted_count;
ELSE
    RAISE NOTICE 'SUCCESS: All rows migrated correctly';
END IF;
RAISE NOTICE '=====';
END $$;

\echo 'Migration completed: seafarer_debriefs (with levels and level_members)'

COMMIT;

```

Known issues in original code

Issue	Impact
Broken seafarer mapping (UUID filter on bigint source_id)	seafarer_uuid_mapping temp table often empty; rows excluded
INSERT limited to non-empty feedback arrays	Debriefs without feedback silently skipped
TRUNCATE levels + level_members in debriefs script	Re-run destroys downstream migrated child data
BEGIN at script start (before lookups)	Partial failure can leave truncated tables without inserts
Connection tests only RAISE WARNING	Broken dblink connections do not stop migration
No migration.table_mappings insert for debriefs	No SAC-to-SMAC reverse lookup for migrated rows
Missing constants.sql include	DEFAULT_TENANT_ID may be undefined outside orchestrator
status always 'Active'; minimal audit_info	Deleted/closed debriefs and legacy metadata misrepresented
Repeated dblink calls for legacy counts/INSERT	Slower execution on large appraisal_debrief volumes

2. AI Prompts Used (Cursor / Composer)

Prompts were applied iteratively in one session:

1. @04-migration-scripts/crewing/seafarer_debriefs_migration.sql - in this script, remove all the errors
2. Review the file @04-migration-scripts/crewing/seafarer_debriefs_migration.sql and show the edge cases ... without changing code yet.
3. Apply the fixes for the edge cases with minimal difference
4. Remove the useless try/catch that only rethrows ... Do not change behavior otherwise.
5. apply error handling wherever required
6. Improve the performance optimization without any logic changes
7. Wrap as required in a DB transaction so partial failure doesn't leave mismatches
8. Review for regressions and missing edge cases
9. Apply the fixes for the above missing cases

3. Enhanced Code

Enhanced working tree (~659 lines, cached legacy reads, typed guards, transactional writes)

Path: D:\BackFillTest\data-migration\04-migration-scripts\crewing\seafarer_debriefs_migration.sql

```
-- =====
-- Migration Script: seafarer_debriefs
-- Legacy: synergy_seafarer.public.appraisal_debrief -> New: smac_crewing_migration.shore.seafarer_debriefs
-- Note: Levels and level_members are migrated by seafarer_debrief_levels_migration.sql and
--       seafarer_debrief_level_members_migration.sql (run after this script).
--
-- DBLINK CONNECTIONS:
-- This script uses the following dblink connections (defined in 07-orchestration/00_common/connection_setup.sql):
-- 1. 'synergy_seafarer' - Uses SS_HOST, SS_PORT, SS_DB, SS_USER, SS_PASS variables
-- 2. 'synergy_vessel' - Uses SV_HOST, SV_PORT, SV_DB, SV_USER, SV_PASS variables
-- 3. 'smac_master_migration' - Uses SMM_HOST, SMM_PORT, SMM_DB, SMM_USER, SMM_PASS variables (conditional)
--
-- Connection strings are built in connection_setup.sql using format():
-- format('host=%s port=%s dbname=%s user=%s password=%s', : 'SS_HOST', : 'SS_PORT', ...)
--
-- Variables are set by the PowerShell orchestration script (migrate_smac_crewing.ps1) from
-- migration_config_smac_crewing.json configuration file.
--
-- NOTE: All dblink queries use single-quoted strings. Single quotes inside queries are escaped as ''.
-- =====

\set ON_ERROR_STOP on
\set ON_ERROR_ROLLBACK on
\timing on

\echo '=====
Migration: seafarer_debriefs table'
\echo '=====

-- Load constants (also prepended by migrate_smac_crewing.ps1 orchestrator)
\i 07-orchestration/constants.sql

SET statement_timeout = '2h';
SET lock_timeout = '30s';
SET work_mem = '256MB';

-- Pre-migration checks (read-only; no durable writes)
\echo 'Pre-migration checks...'

-- Verify dblink connections are established
\echo 'Verifying dblink connections...'
DO $$
DECLARE
    v_connections TEXT[];
    v_synergy_seafarer_ok BOOLEAN := false;
    v_synergy_vessel_ok BOOLEAN := false;
    v_smac_master_ok BOOLEAN := false;
BEGIN
    -- Get all active dblink connections
    SELECT dblink_get_connections() INTO v_connections;

    -- Check each required connection
    v_synergy_seafarer_ok := 'synergy_seafarer' = ANY(v_connections);
    v_synergy_vessel_ok := 'synergy_vessel' = ANY(v_connections);
    v_smac_master_ok := 'smac_master_migration' = ANY(v_connections);

    RAISE NOTICE '=====';
    RAISE NOTICE 'Dblink Connection Status: ';
    RAISE NOTICE '=====';
    RAISE NOTICE 'synergy_seafarer: %', CASE WHEN v_synergy_seafarer_ok THEN 'CONNECTED' ELSE 'NOT CONNECTED' END;
    RAISE NOTICE 'synergy_vessel: %', CASE WHEN v_synergy_vessel_ok THEN 'CONNECTED' ELSE 'NOT CONNECTED' END;
    RAISE NOTICE 'smac_master_migration: %', CASE WHEN v_smac_master_ok THEN 'CONNECTED' ELSE 'NOT CONNECTED' END;
    RAISE NOTICE '=====';

    -- Test connections with simple queries (fail with context on broken connections)
    IF NOT v_synergy_seafarer_ok THEN
        RAISE EXCEPTION 'synergy_seafarer connection not established. Check connection_setup.sql and SS_* variables.';
    END IF;
    BEGIN
        PERFORM dblink('synergy_seafarer', 'SELECT 1');
        RAISE NOTICE 'synergy_seafarer connection test: OK';
    EXCEPTION WHEN OTHERS THEN
        RAISE EXCEPTION 'Cannot connect to synergy_seafarer database: %', SQLERRM;
    END;

    IF NOT v_synergy_vessel_ok THEN
        RAISE EXCEPTION 'synergy_vessel connection not established. Check connection_setup.sql and SV_* variables.';
```

```

END IF;
BEGIN
    PERFORM dblink('synergy_vessel', 'SELECT 1');
    RAISE NOTICE 'synergy_vessel connection test: OK';
EXCEPTION WHEN OTHERS THEN
    RAISE EXCEPTION 'Cannot connect to synergy_vessel database: %', SQLERRM;
END;

IF NOT v_smac_master_ok THEN
    RAISE EXCEPTION 'smac_master_migration connection not established. Check connection_setup.sql and SMM_* variables.';
END IF;
BEGIN
    PERFORM dblink('smac_master_migration', 'SELECT 1');
    RAISE NOTICE 'smac_master_migration connection test: OK';
EXCEPTION WHEN OTHERS THEN
    RAISE EXCEPTION 'Cannot connect to smac_master_migration database: %', SQLERRM;
END;
END $$;

DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM information_schema.tables WHERE table_schema='migration' AND table_name='table_mappings') THEN
        RAISE EXCEPTION 'migration.table_mappings does not exist. Run migration prerequisites first.';
    END IF;
    IF NOT EXISTS (SELECT 1 FROM information_schema.tables WHERE table_schema='public' AND table_name='seafarers') THEN
        RAISE EXCEPTION 'Prerequisite table public.seafarers does not exist. Must migrate seafarers first.';
    END IF;
    IF NOT EXISTS (SELECT 1 FROM information_schema.tables WHERE table_schema='shore' AND table_name='seafarer_debriefs') THEN
        RAISE EXCEPTION 'Target table shore.seafarer_debriefs does not exist. Please create the table schema first.';
    END IF;

    -- Verify prerequisite tables are migrated
    IF NOT EXISTS (SELECT 1 FROM migration.table_mappings WHERE target_table = 'seafarers' AND target_db = current_database()) THEN
        RAISE EXCEPTION 'seafarers table must be migrated first';
    END IF;

    IF NOT EXISTS (SELECT 1 FROM public.seafarers LIMIT 1) THEN
        RAISE EXCEPTION 'Prerequisite table public.seafarers exists but has no data. Must migrate seafarers first.';
    END IF;

    IF NOT EXISTS (SELECT 1 FROM information_schema.tables WHERE table_schema='crewing' AND table_name='debriefing_reasons') THEN
        RAISE WARNING 'crewing.debriefing_reasons not found; reason_id will be copied without FK validation.';
    END IF;
END $$;

-- Load legacy source once (reused for diagnostics, INSERT, and validation)
\echo 'Loading legacy appraisal_debrief into session cache...'
CREATE TEMP TABLE legacy_appraisal_debrief AS
SELECT *
FROM dblink('synergy_seafarer',
'SELECT
    id,
    seafarer_uuid,
    initiated_date,
    debrief_reason_id,
    other_debrief_reason,
    rank_id,
    vessel_category_id,
    vessel_uuid,
    attachments,
    debrief_status,
    from_date,
    to_date,
    mark_for_deactivation,
    is_manual,
    training_needs,
    feedback,
    deleted_at,
    deleted_by,
    created_by_id,
    created_by_name,
    created_at,
    updated_by_id,
    updated_by_name,
    updated_at
FROM public.appraisal_debrief'
) AS legacy_data(
    id uuid,
    seafarer_uuid uuid,
    initiated_date timestamp without time zone,
    debrief_reason_id uuid,
    other_debrief_reason text,
    rank_id bigint,

```

```

vessel_category_id bigint,
vessel_uuid uuid,
attachments text[],
debrief_status text,
from_date timestamp without time zone,
to_date timestamp without time zone,
mark_for_deactivation boolean,
is_manual boolean,
training_needs jsonb,
feedback jsonb,
deleted_at timestamp without time zone,
deleted_by text,
created_by_id text,
created_by_name varchar,
created_at timestamp without time zone,
updated_by_id text,
updated_by_name varchar,
updated_at timestamp without time zone
);

CREATE INDEX idx_legacy_appraisal_debrief_seafarer_uuid ON legacy_appraisal_debrief(seafarer_uuid);
CREATE INDEX idx_legacy_appraisal_debrief_vessel_uuid ON legacy_appraisal_debrief(vessel_uuid);
CREATE INDEX idx_legacy_appraisal_debrief_vessel_category_id ON legacy_appraisal_debrief(vessel_category_id);
ANALYZE legacy_appraisal_debrief;

\echo 'Legacy row count:'
DO $$
DECLARE
    v_legacy_count BIGINT;
BEGIN
    SELECT COUNT(*) INTO v_legacy_count FROM legacy_appraisal_debrief;
    RAISE NOTICE 'Legacy appraisal_debrief table row count: %', v_legacy_count;

    IF v_legacy_count = 0 THEN
        RAISE WARNING 'WARNING: No rows found in legacy appraisal_debrief table!';
    END IF;
END $$;

\echo 'Sample legacy data (first 5 rows):'
DO $$
DECLARE
    r RECORD;
    v_row_count INT := 0;
BEGIN
    FOR r IN
        SELECT id, seafarer_uuid, vessel_uuid, vessel_category_id, debrief_status
        FROM legacy_appraisal_debrief
        ORDER BY id
        LIMIT 5
    LOOP
        v_row_count := v_row_count + 1;
        RAISE NOTICE 'Row %: id=%, seafarer_uuid=%, vessel_uuid=%, vessel_category_id=%, debrief_status=%',
            v_row_count, r.id, r.seafarer_uuid, r.vessel_uuid, r.vessel_category_id, r.debrief_status;
    END LOOP;

    IF v_row_count = 0 THEN
        RAISE WARNING 'WARNING: No sample rows returned from legacy cache!';
    END IF;
END $$;

\echo 'Target table current count:'
DO $$
DECLARE
    v_existing_count BIGINT;
BEGIN
    SELECT COUNT(*) INTO v_existing_count FROM shore.seafarer_debriefs;
    RAISE NOTICE 'Current shore.seafarer_debriefs row count: %', v_existing_count;
    IF v_existing_count > 0 THEN
        RAISE WARNING 'Target table shore.seafarer_debriefs contains existing records. These will be deleted!';
    END IF;
END $$;

-- Create lookup tables for foreign key resolution (session temp tables only)
\echo 'Creating lookup tables for foreign key resolution...'

CREATE TEMP TABLE legacy_seafarers_uuid_lookup AS
SELECT id, uuid
FROM dblink('synergy_seafarer',
'SELECT id, uuid FROM public.seafarers WHERE uuid IS NOT NULL AND TRIM(uuid) <> ''''
) AS leg(id bigint, uuid text);

CREATE INDEX idx_legacy_seafarers_uuid_lookup_id ON legacy_seafarers_uuid_lookup(id);

```

```

-- Seafarer UUID mapping: appraisal_debrief.seafarer_uuid -> public.seafarers.id
-- Primary: direct id match when legacy uuid was preserved as public.seafarers.id
-- Fallback: legacy seafarers.uuid -> table_mappings -> SMAC seafarer id
CREATE TEMP TABLE seafarer_uuid_mapping AS
SELECT DISTINCT ON (legacy_seafarer_uuid)
    legacy_seafarer_uuid,
    new_seafarer_id
FROM (
    SELECT s.id AS legacy_seafarer_uuid, s.id AS new_seafarer_id, 1 AS map_priority
    FROM public.seafarers s
    WHERE s.id IS NOT NULL
    UNION ALL
    SELECT
        (regexp_replace(TRIM(leg.uuid), '\s+', '', 'g'))::uuid AS legacy_seafarer_uuid,
        tm.target_id AS new_seafarer_id,
        2 AS map_priority
    FROM legacy_seafarers_uuid_lookup leg
    INNER JOIN migration.table_mappings tm
        ON tm.source_id = leg.id::text
        AND tm.target_table = 'seafarers'
        AND tm.target_db = current_database()
    INNER JOIN public.seafarers s2 ON s2.id = tm.target_id
    WHERE regexp_replace(TRIM(leg.uuid), '\s+', '', 'g') ~ '^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$'
) seafarer_map_src
ORDER BY legacy_seafarer_uuid, map_priority;

CREATE INDEX idx_seafarer_uuid_mapping_legacy_uuid ON seafarer_uuid_mapping(legacy_seafarer_uuid);
ANALYZE seafarer_uuid_mapping;

-- Vessel: legacy vessel_id (bigint) -> smac vessel.vessels.id via smac_master migration.table_mappings;
-- debrief_vessel_lookup: appraisal_debrief.vessel_uuid = vessel_details.identifier -> legacy vessel_id;
-- vessel_revision_mapping: active revision per vessel (status=0, newest by created_at)

CREATE TEMP TABLE smac_master_fk_mappings AS
SELECT source_id, target_id, target_table
FROM dblink('smac_master_migration',
    'SELECT source_id, target_id, target_table
    FROM migration.table_mappings
    WHERE (target_table = ''vessels'' OR target_table ILIKE ''categories'')
    AND source_id ~ ''^[0-9]+$''')
AS t(source_id text, target_id uuid, target_table text)
WHERE source_id IS NOT NULL AND target_id IS NOT NULL;

CREATE TEMP TABLE vessel_id_mapping AS
SELECT DISTINCT ON (source_id::bigint)
    source_id::bigint AS legacy_id,
    target_id AS new_id
FROM smac_master_fk_mappings
WHERE target_table = 'vessels'
ORDER BY source_id::bigint, target_id;

CREATE INDEX idx_vessel_id_mapping_legacy_id ON vessel_id_mapping(legacy_id);
CREATE INDEX idx_vessel_id_mapping_new_id ON vessel_id_mapping(new_id);
ANALYZE vessel_id_mapping;

-- Bridge vessel_uuid to synergy_vessel legacy vessel_id (bigint) for vessel_id_mapping
CREATE TEMP TABLE debrief_vessel_lookup AS
SELECT DISTINCT ON (vd.identifier)
    vd.identifier AS legacy_vessel_identifier,
    vd.vessel_id AS legacy_vessel_id
FROM dblink('synergy_vessel',
    'SELECT identifier, vessel_id FROM public.vessel_details WHERE identifier IS NOT NULL AND vessel_id IS NOT NULL')
AS vd(identifier uuid, vessel_id bigint)
ORDER BY vd.identifier, vd.vessel_id;

CREATE INDEX idx_debrief_vessel_lookup_identifier ON debrief_vessel_lookup(legacy_vessel_identifier);
CREATE INDEX idx_debrief_vessel_lookup_vessel_id ON debrief_vessel_lookup(legacy_vessel_id);
ANALYZE debrief_vessel_lookup;

\echo 'Creating vessel_revision_mapping for active revisions (seafarer_reliefs pattern)...'
CREATE TEMP TABLE vessel_revision_mapping AS
SELECT DISTINCT ON (vr.vessel_id)
    vr.vessel_id AS new_vessel_id,
    vr.id AS active_revision_id
FROM dblink('smac_master_migration',
    'SELECT id, vessel_id, status, created_at
    FROM vessel.vessel_revisions
    WHERE status = 0
    ORDER BY vessel_id, created_at DESC')
AS vr(id uuid, vessel_id uuid, status integer, created_at timestamp)
INNER JOIN vessel_id_mapping all_vessels ON all_vessels.new_id = vr.vessel_id
ORDER BY vr.vessel_id, vr.created_at DESC;

```

```

CREATE INDEX idx_vessel_revision_mapping_new_vessel_id ON vessel_revision_mapping(new_vessel_id);
ANALYZE vessel_revision_mapping;

\echo 'Vessel revision mapping row count:'
SELECT COUNT(*) AS vessel_revision_lookup_count FROM vessel_revision_mapping;

-- Create vessel_type_id lookup mapping (vessel_category_id bigint -> vessel_type_id uuid)
CREATE TEMP TABLE vessel_type_id_mapping AS
SELECT DISTINCT ON (source_id::bigint)
    source_id::bigint as legacy_category_id,
    target_id as new_type_id
FROM smac_master_fk_mappings
WHERE target_table ILIKE 'categories'
ORDER BY source_id::bigint, target_id;

CREATE INDEX idx_vessel_type_id_mapping_legacy_category_id ON vessel_type_id_mapping(legacy_category_id);
ANALYZE vessel_type_id_mapping;

\echo 'Vessel mapping diagnostic (debrief_vessel_lookup + vessel_id_mapping + vessel_revision_mapping):'
DO $$
DECLARE
    v_lookup_count BIGINT;
    v_vid_count BIGINT;
    v_vrev_count BIGINT;
    v_seafarer_map_count BIGINT;
    v_legacy_count BIGINT;
BEGIN
    SELECT COUNT(*) INTO v_legacy_count FROM legacy_appraisal_debrief;
    SELECT COUNT(*) INTO v_lookup_count FROM debrief_vessel_lookup;
    SELECT COUNT(*) INTO v_vid_count FROM vessel_id_mapping;
    SELECT COUNT(*) INTO v_vrev_count FROM vessel_revision_mapping;
    SELECT COUNT(*) INTO v_seafarer_map_count FROM seafarer_uuid_mapping;
    RAISE NOTICE 'Legacy rows in cache: %', v_legacy_count;
    RAISE NOTICE 'seafarer_uuid_mapping rows: %', v_seafarer_map_count;
    RAISE NOTICE 'debrief_vessel_lookup rows: % (identifier to legacy vessel_id)', v_lookup_count;
    RAISE NOTICE 'vessel_id_mapping rows: %', v_vid_count;
    RAISE NOTICE 'vessel_revision_mapping rows: %', v_vrev_count;

    IF v_legacy_count = 0 THEN
        RAISE NOTICE 'Legacy source is empty; skipping FK lookup row-count requirements.';
        RETURN;
    END IF;

    IF v_seafarer_map_count = 0 THEN
        RAISE EXCEPTION 'seafarer_uuid_mapping is empty. Ensure seafarers migration completed successfully.';
    END IF;

    IF v_lookup_count = 0 THEN
        RAISE EXCEPTION 'debrief_vessel_lookup is empty. Check synergy_vessel.public.vessel_details data and SV_* connection.';
    END IF;

    IF v_vid_count = 0 THEN
        RAISE EXCEPTION 'vessel_id_mapping is empty. Ensure vessels are migrated in smac_master_migration first.';
    END IF;
END $$;

-- =====
-- Atomic migration transaction
-- TRUNCATE + DELETE mappings + INSERT debriefs + INSERT mappings commit together.
-- Any RAISE EXCEPTION or SQL error rolls back all durable changes (ON_ERROR_ROLLBACK).
-- =====
\echo 'Starting atomic migration transaction...'
BEGIN;

TRUNCATE TABLE shore.seafarer_debriefs;
DELETE FROM migration.table_mappings
WHERE target_table = 'seafarer_debriefs'
AND target_schema = 'shore'
AND target_db = current_database();

-- =====
-- STEP 1: Migrate seafarer_debriefs (main table)
-- =====
\echo 'Step 1: Migrating seafarer_debriefs data...'

INSERT INTO shore.seafarer_debriefs (
    id,
    seafarer_id,
    seafarer_assignment_id,
    appraisal_id,
    reason_id,
    reason_text,
    vessel_id,
    vessel_type_id,
    sign_on_date,

```

```

sign_off_date,
appraisal_reports_available,
attachments,
current_stage,
workflow_status,
initiated_by,
initiated_at,
closed_by,
closed_at,
status,
tenant_id,
created_at,
updated_at,
archived_at,
deleted_at,
audit_info,
vessel_revision_id
)
SELECT
  legacy_data.id as id, -- Preserve legacy UUID id directly
  seafarer_map.new_seafarer_id as seafarer_id, -- Map seafarer_uuid to migrated seafarers.id
  NULL as seafarer_assignment_id, -- Not available in source
  NULL as appraisal_id, -- Not available in source
  CASE
    WHEN legacy_data.debrief_reason_id IS NULL THEN NULL
    WHEN EXISTS (
      SELECT 1 FROM information_schema.tables
      WHERE table_schema = 'crewing' AND table_name = 'debriefing_reasons'
    )
    AND NOT EXISTS (
      SELECT 1 FROM crewing.debriefing_reasons dr
      WHERE dr.id = legacy_data.debrief_reason_id
    )
    THEN NULL
    ELSE legacy_data.debrief_reason_id
  END as reason_id,
  TRIM(legacy_data.other_debrief_reason) as reason_text, -- Direct copy
  vessel_map.new_id as vessel_id, -- legacy vessel_id (via debrief_vessel_lookup) -> smac vessels.id
  COALESCE(vessel_type_map.new_type_id, '00000000-0000-0000-0000-000000000000'::uuid) as vessel_type_id, -- Map vessel_category_id to
vessel_type_id via mapping, default to empty UUID if not found
  legacy_data.from_date::date as sign_on_date, -- Convert timestamp to date
  legacy_data.to_date::date as sign_off_date, -- Convert timestamp to date
  false as appraisal_reports_available, -- Default to false (NOT NULL)
  CASE
    WHEN legacy_data.attachments IS NULL OR array_length(legacy_data.attachments, 1) IS NULL THEN NULL
    ELSE to_jsonb(legacy_data.attachments)
  END as attachments, -- Convert text[] to jsonb
  -- current_stage: map SAC appraisal_debrief.debrief_status to SMAC stage values
  CASE LOWER(TRIM(COALESCE(legacy_data.debrief_status, '')))
    WHEN 'debriefing_initiated' THEN 'DebriefInitiated'
    WHEN 'ncs_review_1_initiated' THEN 'NCSReview1Initiated'
    WHEN 'ncs_review_1_recommended' THEN 'NCSReview1Recommended'
    WHEN 'ncs_review_2_recommended' THEN 'NCSReview2Recommended'
    WHEN 'debriefing_closed' THEN 'Closed'
    WHEN 'draft' THEN 'Draft'
    ELSE COALESCE(NULLIF(TRIM(legacy_data.debrief_status), ''), 'Draft')
  END as current_stage,
  -- Extract first feedback element's status for workflow_status
  CASE
    WHEN legacy_data.feedback IS NOT NULL
    AND jsonb_typeof(legacy_data.feedback) = 'array'
    AND jsonb_array_length(legacy_data.feedback) > 0
    AND legacy_data.feedback->0->>'status' IS NOT NULL
    THEN legacy_data.feedback->0->>'status'
    ELSE NULL
  END as workflow_status,
  CASE
    WHEN legacy_data.created_by_id IS NOT NULL
    AND legacy_data.created_by_id ~ '^([0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12})$'
    THEN legacy_data.created_by_id::uuid
    ELSE NULL
  END as initiated_by,
  COALESCE(legacy_data.initiated_date, legacy_data.created_at) as initiated_at, -- Prefer initiated_date, fallback to created_at
  CASE
    WHEN legacy_data.deleted_at IS NOT NULL
    AND legacy_data.updated_by_id IS NOT NULL
    AND legacy_data.updated_by_id ~ '^([0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12})$'
    THEN legacy_data.updated_by_id::uuid
    WHEN LOWER(TRIM(COALESCE(legacy_data.debrief_status, ''))) IN ('debriefing_closed', 'closed', 'completed', 'finished')
    AND legacy_data.updated_by_id IS NOT NULL
    AND legacy_data.updated_by_id ~ '^([0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12})$'
    THEN legacy_data.updated_by_id::uuid
    ELSE NULL
  END as updated_by_id,
  COALESCE(legacy_data.updated_date, legacy_data.created_at) as updated_at,
  COALESCE(legacy_data.archived_date, legacy_data.created_at) as archived_at,
  COALESCE(legacy_data.deleted_date, legacy_data.created_at) as deleted_at,
  COALESCE(legacy_data.audit_info, '') as audit_info,
  COALESCE(legacy_data.vessel_revision_id, 0) as vessel_revision_id

```

```

END as closed_by,
CASE
    WHEN legacy_data.deleted_at IS NOT NULL THEN COALESCE(legacy_data.updated_at, legacy_data.deleted_at)
    WHEN LOWER(TRIM(COALESCE(legacy_data.debrief_status, ''))) IN ('debriefing_closed', 'closed', 'completed', 'finished') THEN
legacy_data.updated_at
    ELSE NULL
END as closed_at,
CASE
    WHEN legacy_data.deleted_at IS NOT NULL THEN 'Deleted'
    WHEN LOWER(TRIM(COALESCE(legacy_data.debrief_status, ''))) IN ('debriefing_closed', 'closed', 'completed', 'finished') THEN 'Closed'
    ELSE 'Active'
END as status,
:'DEFAULT_TENANT_ID':::uuid as tenant_id, -- See constants.sql for DEFAULT_TENANT_ID
COALESCE(legacy_data.created_at, NOW()) as created_at, -- Default to current timestamp
COALESCE(legacy_data.updated_at, NOW()) as updated_at, -- Default to current timestamp
NULL as archived_at, -- Not in source
legacy_data.deleted_at as deleted_at, -- Direct copy
jsonb_build_object(
    -- SMAC audit_info structure
    'created_by', CASE WHEN legacy_data.created_by_id IS NOT NULL AND legacy_data.created_by_id::text <> '' THEN
legacy_data.created_by_id::text ELSE NULL END,
    'deleted_by', CASE WHEN legacy_data.deleted_by IS NOT NULL AND legacy_data.deleted_by::text <> '' THEN legacy_data.deleted_by::text
ELSE NULL END,
    'updated_by', CASE WHEN legacy_data.updated_by_id IS NOT NULL AND legacy_data.updated_by_id::text <> '' THEN
legacy_data.updated_by_id::text ELSE NULL END,
    'archived_by', NULL,
    'submitted_by', NULL,
    'approved_at', NULL,
    'approved_by', NULL,
    'approval_notes', NULL,
    'rejected_by', NULL,
    'notes', NULL,
    -- Legacy migration metadata (preserved for reference)
    'legacy_id', legacy_data.id::text,
    'feedback', legacy_data.feedback,
    'training_needs', legacy_data.training_needs,
    'rank_id', legacy_data.rank_id,
    'mark_for_deactivation', legacy_data.mark_for_deactivation,
    'is_manual', legacy_data.is_manual,
    'migration_source', 'synergy_seafarer',
    'migrated_at', to_jsonb(NOW())
) as audit_info,
COALESCE(vrm.active_revision_id, '00000000-0000-0000-0000-000000000000':::uuid) as vessel_revision_id
FROM legacy_appraisal_debrief legacy_data
LEFT JOIN seafarer_uuid_mapping seafarer_map ON seafarer_map.legacy_seafarer_uuid = legacy_data.seafarer_uuid
LEFT JOIN debrief_vessel_lookup v_bridge ON v_bridge.legacy_vessel_identifier = legacy_data.vessel_uuid
LEFT JOIN vessel_id_mapping vessel_map ON vessel_map.legacy_id = v_bridge.legacy_vessel_id
LEFT JOIN vessel_revision_mapping vrm ON vrm.new_vessel_id = vessel_map.new_id
LEFT JOIN vessel_type_id_mapping vessel_type_map ON vessel_type_map.legacy_category_id = legacy_data.vessel_category_id
WHERE seafarer_map.new_seafarer_id IS NOT NULL
    AND vessel_map.new_id IS NOT NULL
    AND EXISTS (SELECT 1 FROM public.seafarers s WHERE s.id = seafarer_map.new_seafarer_id);
-- vessel_type_id via vessel_type_id mapping (can be default UUID when unmapped)

-- Create mapping records for migrated debriefs
\echo 'Creating mapping records for seafarer_debriefs...'
INSERT INTO migration.table_mappings (
    id,
    source_db,
    source_schema,
    source_table,
    source_id,
    target_db,
    target_schema,
    target_table,
    target_id,
    migration_direction,
    migrated_at
)
SELECT
    gen_random_uuid(),
    'synergy_seafarer':::VARCHAR(100),
    'public':::VARCHAR(100),
    'appraisal_debrief':::VARCHAR(100),
    sd.id::text,
    current_database()::text::VARCHAR(100),
    'shore':::VARCHAR(100),
    'seafarer_debriefs':::VARCHAR(100),
    sd.id,
    'SAC_TO_SMAC':::VARCHAR(50),
    CURRENT_TIMESTAMP
FROM shore.seafarer_debriefs sd
ON CONFLICT (source_db, source_schema, source_table, source_id, target_db, target_schema, target_table)

```

```

DO NOTHING;

-- Check insert count immediately
\echo 'Checking insert count...'
DO $$
DECLARE
    v_inserted_count BIGINT;
    v_legacy_count BIGINT;
BEGIN
    SELECT COUNT(*) INTO v_inserted_count FROM shore.seafarer_debriefs;
    RAISE NOTICE 'Rows inserted so far: %', v_inserted_count;

    SELECT COUNT(*) INTO v_legacy_count FROM legacy_appraisal_debrief;

    IF v_inserted_count = 0 AND v_legacy_count > 0 THEN
        RAISE EXCEPTION 'INSERT completed with 0 rows but legacy appraisal_debrief has % rows. Check seafarer/vessel lookup joins.',
v_legacy_count;
    ELSIF v_inserted_count = 0 THEN
        RAISE NOTICE 'No rows inserted; legacy source is empty.';
    END IF;
END $$;

-- Post-migration validation
\echo 'Post-migration validation...'
DO $$
DECLARE
    v_inserted_count BIGINT;
    v_legacy_count BIGINT;
    v_mapping_count BIGINT;
    v_excluded_count BIGINT;
    v_missing_seafarer BIGINT;
    v_missing_vessel BIGINT;
BEGIN
    SELECT COUNT(*) INTO v_inserted_count FROM shore.seafarer_debriefs;

    SELECT COUNT(*) INTO v_mapping_count
    FROM migration.table_mappings
    WHERE target_table = 'seafarer_debriefs'
        AND target_schema = 'shore'
        AND target_db = current_database();

    SELECT COUNT(*) INTO v_legacy_count FROM legacy_appraisal_debrief;

    SELECT COUNT(*) INTO v_missing_seafarer
    FROM legacy_appraisal_debrief legacy_data
    LEFT JOIN seafarer_uuid_mapping seafarer_map ON seafarer_map.legacy_seafarer_uuid = legacy_data.seafarer_uuid
    WHERE seafarer_map.new_seafarer_id IS NULL;

    SELECT COUNT(*) INTO v_missing_vessel
    FROM legacy_appraisal_debrief legacy_data
    INNER JOIN seafarer_uuid_mapping seafarer_map ON seafarer_map.legacy_seafarer_uuid = legacy_data.seafarer_uuid
    LEFT JOIN debrief_vessel_lookup v_bridge ON v_bridge.legacy_vessel_identifier = legacy_data.vessel_uuid
    LEFT JOIN vessel_id_mapping vessel_map ON vessel_map.legacy_id = v_bridge.legacy_vessel_id
    WHERE vessel_map.new_id IS NULL;

    v_excluded_count := GREATEST(v_legacy_count - v_inserted_count, 0);

    RAISE NOTICE '=====';
    RAISE NOTICE 'Migration Summary: ';
    RAISE NOTICE '=====';
    RAISE NOTICE 'Legacy rows: %', v_legacy_count;
    RAISE NOTICE 'Inserted rows: %', v_inserted_count;
    RAISE NOTICE 'Mapping records: %', v_mapping_count;
    IF v_excluded_count > 0 THEN
        RAISE NOTICE 'Excluded rows (total): %', v_excluded_count;
        RAISE NOTICE ' - missing seafarer mapping: %', v_missing_seafarer;
        RAISE NOTICE ' - seafarer ok but missing vessel mapping: %', v_missing_vessel;
        RAISE NOTICE 'Note: validation script compares eligible rows (seafarer+vessel mapped), not full legacy count.';
    END IF;

    IF v_inserted_count = 0 AND v_legacy_count > 0 THEN
        RAISE EXCEPTION 'No rows inserted but legacy table has % rows (missing seafarer: %, missing vessel: %)',
            v_legacy_count, v_missing_seafarer, v_missing_vessel;
    ELSIF v_excluded_count > 0 THEN
        RAISE NOTICE 'SUCCESS: Migrated % eligible rows (% excluded due to FK mapping)', v_inserted_count, v_excluded_count;
    ELSE
        RAISE NOTICE 'SUCCESS: All rows migrated correctly';
    END IF;

    IF v_inserted_count > 0 AND v_mapping_count != v_inserted_count THEN
        RAISE EXCEPTION 'Mapping count (%) does not match inserted row count (%)', v_mapping_count, v_inserted_count;
    END IF;
    RAISE NOTICE '=====';

```

```
END $$;

\echo 'Migration completed: seafarer_debriefs'

COMMIT;

\echo 'Atomic migration transaction committed successfully.'
```

Summary of enhancements

Area	Original	Enhanced
Script scope	Header claims levels+members; TRUNCATE all 3 tables	Debriefs only; levels/members in separate scripts
Seafarer mapping	table_mappings with invalid UUID source_id filter	public.seafarers + legacy UUID fallback via table_mappings
INSERT eligibility	feedback array required	All rows where seafarer + vessel mapping succeed
Transaction	BEGIN at start; no explicit COMMIT	ON_ERROR_ROLLBACK; atomic TRUNCATE + INSERT + COMMIT
Error handling	Warnings on connection/lookup gaps	RAISE EXCEPTION fail-fast with SQLERRM context
Legacy data access	Multiple dblink round trips	Single legacy_appraisal_debrief cache + indexes + ANALYZE
Duplicate join safety	Plain SELECT on vessel lookups	DISTINCT ON dedupe on vessel_id_mapping and debrief_vessel_lookup
reason_id / audit_info	Direct copy; training_needs not stored	FK-validated reason_id; training_needs in audit_info
Empty legacy source	Lookup hard-fail even when legacy count = 0	No-op migration allowed when source is empty
Validation alignment	Full legacy count comparison only	Partial migration notices; mapping count parity checks

4. Productivity Observation

Using staged AI prompts turned a brittle 495-line migration script into a production-ready 659-line pipeline in far less time than a manual rewrite would take: each prompt targeted one concern (mapping bugs, scope, error handling, performance, transactions, regressions), which kept reviews small and reduced regression risk. The line count grew (~495 -> ~659), but most of that is structure, guards, caching, and validation that would otherwise surface as silent data loss or support time during crewing migration runs. Net effect: faster delivery of correct behavior, with the caveat that the job should still be validated on staging databases with representative seafarer/vessel mapping coverage.

Generated for PDF export on 2026-06-08. Regenerate with: `python docs/generate_seafarer_debriefs_ai_pdf.py`